Dr. Wittawin Susutti

wittawin.sus@kmutt.ac.th

CSS234 WEB PROGRAMMING II

LECTURE 04 ORM

What is ORM?

- A programming technique for converting data between incompatible type systems using object-oriented programming languages.

- It creates a "virtual object database" that can be used from within the programming language.

The Problem: Impedance Mismatch

Relational Model

- Tables, Rows, Foreign Keys, Normalization.

Object Model

- Classes, Inheritance, Composition, Encapsulation.

- Conflict in data representation and relationships.

Life Without ORM (The Hard Way)

- Writing raw SQL strings in code.
- Manual mapping
  - Extracting values from `ResultSet` to Objects.
- Risk of SQL Injection if not handled carefully.

Differences between ORM and SQL

	ABSTRACTION LEVEL	DATA MODEL	COMPLEXITY	PERFORMANCE	PORTABILITY
ORM Object-Relational Mapping	Higher level of abstraction than SQL, works with objects and classes in their programming language	Designed to work with an object-oriented data model	Simpler to use than SQL	Slower to use than SQL, especially for complex queries that involve multiple tables	Less portable than SQL since it is often specific to a programming language or database management system
SQL Structured Query Language	Lower level of abstraction than ORM, involves writing queries using database management system-specific syntax	Designed to work with a relational data model	Harder to use than ORM	Faster to use than ORM, especially for complex queries that involve multiple tables	More portable than ORM since it can be used with any relational database management system

techslang.com

The ORM Layer: An Abstract Bridge

- ORM sits between the application and the database.
- Acts as a translator that speaks both SQL and OOP.

Mapping Fundamentals (1:1 Correspondence)

- Class → Table
- Field/Attribute → Column
- Object Instance → Row

Defining Relationships: One-to-Many

- Example: A User has many Posts.
- ORM handles the Foreign Key logic automatically.
- Objects hold collections (lists) of related objects.

Defining Relationships: Many-to-Many

- Example: Students and Courses.
- Requires a "Junction Table" (Join Table).
- ORM abstracts away the complexity of managing the bridge table.

Database Migrations (The Evolution)

- Tools: **Alembic** (Python), **Prisma Migrate** (JS).
- Keeps DB schema in sync with your code.
- Version control for your database structure.

Lazy vs. Eager Loading

- **Lazy Loading:** Fetch related data only when accessed (can cause N+1 problem).
- **Eager Loading:** Fetch everything in one JOIN query.

N+1 Problem

The Concept

- The N+1 problem occurs when an application makes **1** initial database query to fetch a list of parent objects, and then makes **N** additional queries to fetch related child objects for *each* parent.

The Math

- Total Queries = 1 (Parent List) + N (One per child relation).

Classic Example

- A "Blog" application.
  - Fetch 100 Articles (`SELECT * FROM articles`).
  - Loop through each article to display the author's name.
  - Result: 101 Database queries executed.

The Root Cause: Lazy Loading

ORM Behavior

- Most ORMs (Hibernate, Entity Framework, Eloquent, ActiveRecord) default to **Lazy Loading**.

How it works

- Data is fetched from the database only when the application code explicitly accesses the property/relationship.

The Trap

- Code looks clean: `article.getAuthor().getName()` inside a loop.
- Behind the scenes
  - The ORM triggers a SQL query *inside* the iteration because the Author data wasn't fetched initially.

Performance Impact Analysis

Latency Accumulation

- It's not just about CPU load; it's about **Network Round Trip Time (RTT)**.
- Even if a query takes 1ms, 1000 queries = 1 second of pure network latency (sequential blocking).

Database Connection Pool Exhaustion

- High volume of small queries can hog connections, preventing other legitimate requests from being processed.

The "Death by a Thousand Cuts"

- Small, fast queries accumulating to cause significant page load delays.

Eager Loading & Batching

Solution 1: Eager Loading (Joins)

- Fetch all necessary data in the *initial* query using `JOIN`.
- *SQL:* `SELECT * FROM articles JOIN authors ON ...`
- *ORM Syntax:* `.include(:author) (Rails), .with('author') (Laravel), JOIN FETCH (JPA/Hibernate).`
- *Result:* 1 Query total.

Solution 2: Batching (The IN Clause)

- Fetch parents first. Collect all Parent IDs.
- Fetch all children in one go: `SELECT * FROM authors WHERE id IN (1, 2, 3, ...)`
- *Result:* 2 Queries total (1 for parents, 1 for all children).

Detection & Tools

Log Inspection

- Enable SQL logging in development/staging (e.g., show-sql in Hibernate/Spring).
- Look for repetitive patterns `SELECT ... WHERE id = ?`.

Automated Tools

- **Ruby on Rails:** bullet gem (alerts you when N+1 is detected).
- **Java/Hibernate:** datasource-proxy or Hibernate Statistics.
- **APM Tools:** New Relic, Datadog, Sentry (often highlight "N+1" or "Too many DB calls").

Testing

- Write integration tests that assert the number of queries executed.

Advantages

DRY (Don't Repeat Yourself)

- Define the model once.

Maintainability

- Easier to refactor code.

Vendor Agnostic

- Swap MySQL for Postgres easily.

Disadvantages

Performance Overhead

- Extra layer takes time.

Complex Queries

- Raw SQL is often better for 20-table JOINs.

Abstraction Leakage

- You still need to understand SQL to debug.

Security

Preventing SQL Injection

- ORMs use **Parameterized Queries** by default.
- Automatically escapes dangerous characters.

Best Practices

- Don't use ORM for everything (e.g., Bulk Inserts).
- Keep models lean; move logic to services.
- Always profile your queries in development.

Database Schema Migration

Definition

- Schema migration is the management of incremental, reversible changes to relational database schemas.

Why it matters

  - Aligns database structure with application code evolution.
  - Provides version control for the database (similar to Git for code).

Core Principle

- Database schema should not be changed manually (e.g., via SSH/GUI) in production; it must be automated and reproducible.

Migration Strategies: Offline vs. Online

Offline Migration ("Big Bang")

- Stop the application -> Run migration scripts -> Restart application.
- *Pros:* Simple, ensures data consistency immediately.
- *Cons:* Significant downtime, risky if rollback is needed.

Online Migration (Zero Downtime)

- Database remains live during changes.
- Requires backward-compatible schema changes.
- Decouples deployment of code from migration of database.

The Expand and Contract Pattern

Concept

- Also known as "Parallel Change". A 3-phase process to safely introduce breaking changes.

Phase 1: Expand (Add)

- Add new columns/tables. do NOT remove/rename old ones yet.
- Code writes to *both* old and new columns (Dual-write).

Phase 2: Migrate (Transition)

- Backfill existing data from old to new structures.
- Switch code to read from the new column.

Phase 3: Contract (Remove)

- Remove the old code references.
- Drop the old columns/tables.

Tools & Approaches
Imperative vs. Declarative

Imperative (Migration-based)

- Tools: Flyway, Liquibase, knex.js, Rails Active Record.
- You write specific scripts (e.g., V1__create_user.sql).
- Pros: Precise control over execution.
- Cons: Harder to manage state drift; merging branches can be messy.

Declarative (State-based)

- Tools: **Atlas, Terraform, Prisma Migrate**.
- You define the *desired end state* (schema.hcl/schema.prisma).
- The tool calculates the diff and generates the migration.
- Pros: Easier to read, focuses on the "what" not the "how".

Best Practices & Pitfalls

- **Backup -** Always backup before applying changes.
- **Test in Staging -** Replicate production data (sanitized) to staging to test performance.
- **Transactional DDL -** Use DBs that support transactional schema changes (e.g., PostgreSQL) so failures roll back automatically.
- **Avoid Locking -** Be careful with operations that lock tables for long periods (e.g., adding an index on a large table). Use CONCURRENTLY features if available.
- **Separate Schema & Data Changes -** Don't mix heavy data updates with schema changes in the same transaction.

Lab

- Write a function to create a new **Author** named "J.K. Rowling" and, in the same command, create a **Book** titled "Harry Potter and the Philosopher's Stone" linked to this author.

- Write a function to create a new **Author** named "J.K. Rowling" and, in the same command, create a **Book** titled "Harry Potter and the Philosopher's Stone" linked to this author.

- Create a Category named "Fantasy". 2. Update the book "Harry Potter and the Philosopher's Stone" to link it to the "Fantasy" category.

```
model Author {
  id    Int     @id @default(autoincrement())
  name  String
  bio   String?
  books Book[]
}

model Book {
  id         Int        @id @default(autoincrement())
  title      String
  published  Boolean    @default(false)
  author     Author     @relation(fields: [authorId], references: [id])
  authorId   Int
  categories Category[]
}

model Category {
  id    Int    @id @default(autoincrement())
  name  String @unique
  books Book[]
}
```